

Video Transcripts

Module 3: Artificial Intelligence Technology Fundamentals: Deep Learning

Video 1: Introduction to Deep Learning (01:32)

Last week, we introduced key concepts in machine learning, including ways to categorize different machine learning methods, specifically, supervised, unsupervised, and semi-supervised methods. We discussed parametric and non-parametric methods, as well as probabilistic models.

We will build on these ideas and explore neural networks, and a variety of deep neural network concepts, and architectures that provide great flexibility and power in building AI products. We built intuition about linear classifiers and extended those using a regression and optimization perspective to include logistic and support vector machine classifiers. We considered non-parametric methods, including tree-based approaches. We also introduced probabilistic frameworks, including Bayesian approaches.

As we now turn to neural networks, we'll revisit classification and regression, first in simple structures, including multi-layer perceptrons or MLPs. We'll then build to more complicated, deep structures, including convolutional neural networks or CNNs, and recurrent neural networks or RNNs. We'll see that deep neural networks provide a wide range of architectures, opportunities, and challenges, for learning.

Video 2: Learning to Cure Cancer (06:21)

Currently the average risk for a woman in the United States to develop breast cancer in her lifetime is estimated to be approximately 13%, affecting one in seven women, making it the second most common type of cancer for women. This results in about 2 million cases of breast cancer being diagnosed every year. It is an established fact that directionally, early diagnosis helps reduce both mortality and treatment costs. However, we don't have detailed figures on the value of an earlier detection of breast cancer but there is significant potential for improvement in the detection of small tumors, especially before they expand beyond the breast. There is also evidence that too much screening is negative because of false positive, estimated by a meta-review at about 19%, and the extra cost of additional screening mammographs on a single

woman, which would serve more public good if practiced on a different woman that has not yet been screened.

With tens of millions of screening tests done yearly, one of the most impactful practical decisions of healthcare systems across the world is when to schedule the next mammogram for a given woman. Most often a small number of parameters are used to decide when to do a mammogram. And these may include age range, genetic predisposition, and whether one is a cancer survivor or not. The results become very close to a one-size-fits-all. In fact, often people are divided into course five-year age groups and the same frequency is recommended for everyone on the cohort. Together with her PhD student Adam Yala and others, Professor Regina Barzilay has developed an artificial intelligence system termed Tempo that takes into account many parameters, not just images and age and a few other things. And it's a system that has been trained on a large dataset from MGH, the Massachusetts General Hospital, and tested on hospital data from Emory, Karolinska Sweden, and Chang Gung Memorial Hospitals.

The gist of what they do is very easy to understand, given all the data related to a subject, including past mammograms and other information, they recommend when a new screening should be scheduled. The recommendation is tailored to each person and 100% automatic, as is the subsequent diagnosed produced. For example, they may recommend the schedule of a diagnosis in 18 months and then decide there is an increased risk and recommend the following one after six months, all of these automatically as decided by the algorithm. The results of the research are remarkably positive and, I believe, show a way forward for what is to come with AI in a practical set. Here are the results. Four-months-and-a-half earlier detection on average, think about it. With their system, they detect breast cancer four months ahead of time. Tell a cancer survivor, "Would you have wanted to know four months ahead?" And what's even more remarkable, 25% less mammograms. That is instead of four, three, not bad.

And one of the key insights of Regina's research is that she thinks that mammogram images have much more information than a few lines for an early cancer detection, which is how current practice summarizes the information. That is the mammograms are watched by an expert and summarized, and almost never watched again. And she expresses this by showing some images of breast cancer that look very different from each other, but that have been given the same diagnosis. In one specific example, four different images get the exact same diagnosis, but only three of the people survive and one did not. And one of her goals is to develop systems that can pull out the differences between images, like in the case of the four, and lower mortality and treatment cost by identifying those in higher risk zones. To do this, Regina's model not only uses images, mammograms in this case, but also tries to use patient information, such as genetic prediction pathologies and anything she can find. This is often called multimodal processing

because there are different modes of information being used. One mode being the images, and another can be the genetic information or the information related to other pathologies. Regina's model shows evidence of good prediction of future risk of mammary cancer. So, it's not only useful for early detection, but also for calculating the long-term risk of this particular cancer.

Video 3: Artificial Neural Network (08:15)

The fundamental unit, or basic element, in artificial neural networks is the neuron, based in part on a long and fascinating history of analogies to, or studies of physiological neural brain elements, and networks. For our purposes, a neuron is a mathematical construct. It has some number, m of inputs, x_1 to x_m . Each input has an associated weight, w_i , that multiplies the corresponding input.

These can be pictured as being weights on the arcs leading from the input to the summation unit, indicated with a sigma. These weighted units, or inputs, are summed up together with an offset weight, w_0 , to produce what we call a pre-activation value, z .

Writing this in vector form, we see that the pre-activation, z , is simply a linear function of the inputs, with coefficients w and w_0 . The second part of the neuron, is an activation function, f , that is applied to z to produce an output, a .

We'll return to different kinds of activation functions in a moment, but right now, we note that much of the power of this structure comes from non-linear activation functions. So, that's a single neuron. Additional power comes from assembling multiple neurons into a network.

We can consider a single layer of a neural network is having some number of neurons, or hidden nodes, in the layer. We can arrange n hidden nodes, each with its own set of weights, w and w_0 , its own summation unit, and its own non-linearity or activation function, and its own output, a_i . Importantly, we can fully connect all of the same inputs x_1 to x_n to each of the nodes.

In terms of mathematical notation, we can arrange the weight multiplications and summations in matrix form. This still looks familiar to us as a linear pre-activation, followed by a non-linear function, to produce an output vector, a . So, a single layer of this neural network, can produce a non-linear mapping from vector x to output vector, a .

We see that a neural network is a parametric model, with a known structure. We also see that we can quickly get a large number of model coefficients. For example, if we have 10 inputs, M

equals 10, and 5 hidden nodes, or N equals 5, how many outputs do we have? How many total weights do we have?

Well, the number of outputs is easy. That's just 5, one for each hidden node. For the number of coefficients, we have M plus 1 for each of N nodes, or 10 plus 1 times 5, or 55 model coefficients. Together with the non-linearity, this number of coefficients gives quite a lot of expressive capacity, if we are able to effectively fit, or train, or learn those model coefficients. What tools do we already know for learning model coefficients?

Well, if we can calculate the gradient for any given output, our friend, gradient descent is a powerful method for iteratively learning a good set of coefficients.

Let's refresh ourselves on gradient descent. We start with some cost or objective function J , that tells how big a penalty we have, in terms of overall or average loss or errors for the current coefficients over our dataset. Then, we calculate the gradient of this class function with respect to the model coefficients. That is to say, how much benefit will we have in reducing J for an incremental change in each of the model coefficients. Once we know that, we can do an update to the model coefficients to move downhill on our objective. How fast we move in that direction is governed by the learning rate, etc.

Recall, also stochastic gradient descent, which can be thought of as a randomized, or sample based, version of gradient descent. Here, batches of one or more data points are sampled from our D training, training set, and used to calculate gradient, and then do the model update. We consider some number of batches for each epoch of training over the dataset and continue until some stopping condition on training is met.

Okay, now that we've refreshed ourselves on gradient descent, let's try it out. We'll do this by hand, for a very simple neural network, to build our intuition about how gradient descent works.

Our network has just two inputs, x_1 and x_2 , and a single node. So, a single scalar activation z , and scalar output, y . We'll also consider a very simple activation function, f , which is just the identity, or f of z equals z . We'll also assume some initial weights for our network, w and w_0 , all being 0. Note that in practice, we usually start with random weights to get better training behavior distributed over the network. We want to train this network with respect to a particular loss function, squared loss, or squared error between our prediction, $\hat{y}$ and the actual value, y , for

our training set. We'll also assume a learning rate of 0.1. We have a single training example, x equals 1, 2, with training output, y equals 4. Some questions we want to ask are, what is the current predicted output with our current weights, $\hat{y}$? What are the gradients? What should the new weights be? What is the new prediction with the updated, or new weights?

So, first, the starting output prediction, we simply have, $\hat{y}$ equals 0. Since all the weights were 0. So, our overall error, epsilon is, $\hat{y}$ minus y or -4 . So, not a very good prediction yet. Next, we need the gradient of the overall objective function J equals epsilon squared, with respect to each coefficient or weight. Remembering basic calculus, that is, generally, 2 times epsilon times the gradient of epsilon, with respect to that weight. Since $\hat{y}$ is simply w_0 plus w_1 times x_1 plus w_2 times x_2 , we get a gradient with respect to w_0 of 2 epsilon times one, which for our error, epsilon of -4 , gives us -8 .

We can do the same thing for the gradients with respect to w_1 and w_2 , to get gradients of -8 and -16 . Now, we just update our weights, multiplying those gradients by our learning rate of 0.1, and subtracting. So, now our new weights are 0.8, 0.8, and 1.6. Does this updated model produce a better prediction? We can plug in for our data point. x equals 1, 2, and see that now we predict y equals 4.8, which is much closer to the training value of 4. Maybe that is close enough for our purposes, in which case, we might stop.

Congratulations, you've just trained a neural network by hand.

Video 4: Multi-Layer Perceptrons (MLPs) (05:34)

We've just seen how the basic component of a neural network, the neuron, builds on our previous linear models by adding a non-linear activation function, which expands the space of relationships that can be modeled. The space was further expanded by considering a fully connected neural network layer, consisting of more than one neuron, so that a neural network layer might take in an input x consisting of some number, m features, have some number of hidden nodes, say n , with a corresponding number of pre-activation value, Z , that result from multiplying the inputs x by the weight matrix W and adding offsets W_0 .

That pre-activation for the layer feeds through a non-linearity f , producing n activations in the vector A for that layer. That is already powerful. However, multi-layer perceptron, or MLP structures, build on this even further by having number of sequential layers. Each layer can be

customized in terms of the number of hidden nodes, and the nature of the non-linear activation function.

So, this MLP gives even more power and flexibility to construct highly complex models. I've said that different activation functions can be used. Let me mention a few of the most common and widely used non-linear activation functions.

First, is a familiar one to us, the sigmoid. That function form is what we used when we were doing logistic regression. It maps the Z to the range from 0 to 1, and it's very useful for classification problems, single class, classification, or probability models.

A second non-linear function is the hyperbolic tangent. This maps a little bit differently than the sigmoid in that it maps from -1 to $+1$. This is very useful for regression with both positive and negative values.

Finally, the rectified linear unit, or ReLU, is an interesting non-linearity, which is simply 0 up to some threshold, and then, is linear after that. This is also very useful for both, regression or classification. In addition to customizing the activation function, we can also adapt the loss function at the end of the MLP. The loss function depends on the trained model weights W , and whatever D , data, we're evaluating the loss for. That might be the training loss, or it might be testing loss, if D is our held-out test data, but the nature of the loss function can be customized to the notion of goodness for the model, depending on how it will be used. For example, with regression models, say, predicting continuous values like temperature, squared loss is often used.

On the other hand, for classification problems, where we want to penalize classification errors, the negative log likelihood, as we saw with logistic regression, is common. Having mentioned the loss functions reminds of the question, just how do we train a multi-layer perceptron? The answer is pretty much the same way as with a simple neuron, but with much more complicated, and careful bookkeeping of the calculations. We'll still use supervised training with gradient descent, to find small weight changes that move us downhill, and improve our overall loss and cost function.

Now, the forward calculation for any given x or a batch of x samples, just propagates forward through the network, until we have the output AL , where we compare that output is superscript L , with the training value, Y , that we have and calculate the difference, the loss. We'll also need a

calculation of the gradients of the loss with respect to each and every weight for our particular inputs. Once we have the loss, the so-called, backpropagation step works backward from the end, and apportions the overall loss to the change needed in each weight, to help compensate for the observed error.

So, a lot of calculations involving matrix multiplications, calculation of gradients, and propagation of values through large data structures. Fortunately, machine learning and AI systems like TensorFlow and so on, provide interfaces to set up such networks, and to interface with general or special purpose hardware, that is able to do these calculations amazingly efficiently.

Hopefully, you now have the intuition about what is happening under the covers in neural network training. It's gradient descent and weight adjustments to minimize a loss and objective function.

Video 5: Autoencoders (06:05)

With multi-layer perceptrons, we have the opportunity to build highly flexible neural networks to meet different kinds of needs. We'll consider several architectures, or ways of arranging and customizing neural networks in the next several segments.

The first architecture we'll look at, are autoencoders. Basic idea in an autoencoder is to learn a simplified or encoded representation of a system that does a good job in simply reproducing the inputs to the outputs. The structure is to divide the network in two halves, an encoder, followed by a decoder. The encoder effectively reduces the input down to a simpler, latent space or latent representation. Given that representation of an input, the decoder then seeks to reproduce the outputs.

Unsupervised training is used to learn the encoder and decoder weights to achieve that match. However, the real goal is to create the latent representation. One can think of the encoder as learning, what hopefully small set of features, makes sense to explain or replicate the training data X . For this reason, autoencoders are sometimes used in conjunction with other machine learning methods.

For example, the latent space may be a better representation, that is a good, learned set of features, to use in clustering a dataset. Here is a simple example to illustrate the idea. Here we use the famous MNIST image dataset with many grayscale images of handwritten numeric digits, 0 through 9. This example just looks at digits 0, 1, and 2, where each image is a vector with length

256 of the grayscale pixel values for that image. A simple autoencoder consisting of a single, fully connected encoder, and a single, fully connected decoder is trained on these images with the constraint being to reduce down to a latent representation with just two-dimensions. That is to say, we compress from 256 dimensions, the input grayscale, down to two-dimensions.

We can see that positioned as a function of the a_1 and a_2 latent values, we get surprisingly good separation between groupings of different digits. Conceptually then, the autoencoder is learning to extract two features that might be quite useful in distinguishing or clustering these different digits. While here I've focused on the middle latent representation as the result of the autoencoder architecture, it's important to realize that essentially all neural networks are doing the same thing implicitly as a by-product of its training. Any sophisticated neural network is both trying to learn good features, and then trying to learn how to combine those features in some way to achieve some purpose, such as, classification or prediction.

So, while we may not talk much about latent representations explicitly with other deep learning architectures, it is important intuition to remember that layers of such architecture are often doing one or both of these things: extracting features or combining features implicitly. There is one additional variant of autoencoders worth mentioning before considering other deep learning architectures. Variational autoencoders, or VAEs, combine the neural network idea with a probabilistic representation. In particular, we now want to have our latent representation describe or represent a probability distribution of those feature values.

Typically, one might assume a particular pdf, say a multivariate Gaussian with some mean, μ and covariance structure, σ . The training is done with additional loss terms so that we learn or evolve this latent representation while starting with or staying near to some prior distribution. What this latent representation achieves, is an expanded notion of similarity between different inputs. This latent representation can be used in a rather interesting fashion. In particular, we can randomly sample from this latent pdf, and generate entirely new outputs feeding the sampled a value through just the decoder.

The new outputs thus come from the same distribution as the inputs, and so should have the same important characteristics, but they are entirely fake in that the outputs do not correspond exactly to any one previous training example. This idea, often with more complexity and elaboration, hopefully gives you some intuition about generative machine learning methods that are able to do things like generate fake images of people's faces, or new landscape scenery, and so on.

Video 6: Convolutional Neural Networks (CNNs) (04:46)

Image data is everywhere, and so, perhaps, it should be no surprise that particular neural network architectures have evolved that are particularly customized to handling such data. More generally, when elements of an input, like pixel values in a 2D image have spatial relevance, then convolutional neural networks or CNN structures are a good approach to take advantage of that spatial locality information.

We'll start with a 1D example. Standard signal processing method is a convolution, which can be thought of as a weighted moving average of some input, with the weights being given by a filter. So, for example, we might involve a 1D sequence of 0s and 1s, with a filter of size 2, that has value $-1, 1$. As we shift the filter along the input, we output the product of the filter values with the corresponding shifted inputs.

A CNN layer generally consists of some extra work following the convolution, in particular, the convolution output is typically passed through some non-linear function, often a ReLU. The result of that CNN layer might then be passed to another network layer, such as a fully connected MLP. This same idea is at work with 2D images.

For example, we might have a 9×9 image with grayscale values for each of the 81 pixels. You might convolve that with a 3×3 filter with some weights, such as this structure that sums the outer 8 pixels and subtracts off the inner pixel value. Here we also have the filter move with the so-called stride, or step, of 3 pixels at a time. Thus, the 9×9 input results in a 3×3 convolution output, that then might be fed to a ReLU, with some offset, to produce a final output.

In this particular case, the filter was predesigned to detect a ring-like structure at different locations in the image, and we see that it finds that ring in the middle right side of this image. More generally, the weights of the filter are not known ahead of time. Rather, a big part of the learning and training of the network, is meant to find weights that help with extracting relevant features from the image, wherever they might reside in the image.

For example, our 3×3 filter might have 9 weights. Usually there is an additional offset weight for each filter, as well. If we apply that filter with a stride of 1, and with some 0 buffering at the edges of the input, the convolution in ReLU will result in a 9×9 convolution output. With a stride

of 1, the idea of finding big responses or big matches within regions of the image is called, is accomplished with what are called, max pooling layers.

These just find and output the max value within windows of a given size. Large networks with convolutions often have other variants of these and other layers. For example, we will often have multiple filters in each layer, and may have tensor filters that combine convolutions across multiple channels. To connect up a 2D or tensor result to a fully connected MLP, a flattened layer is often used to map into a 1D vector as needed by the fully connected layer.

Other layers might do things like normalize the values within a layer to keep propagating values in a layer within some range. Indeed, once we start adding these different kinds of layers together into a single neural network, we're well into the territory of building deep neural networks, which we'll consider next.

Video 7: Deep Neural Networks (DNNs) (04:57)

The term, deep neural network, broadly refers to any kind of neural network with many layers, assembled in order to achieve some larger task. For example, we might have an image classification, deep neural network, such as that pictured here.

The input might be three channels, say, red, green, and blue pixel intensities, for a color image. A sequence of convolution plus ReLU, and pooling layers might be assembled, whose purpose is to essentially learn and extract relevant features from the image. That might then be combined with a flattened operation in sequence of fully connected layers culminating in some output.

For example, using a softmax activation function at the end to indicate the best choice among several possible classes for that image. While this architecture can be quite complex, the key enabler in deep learning is still the workhorse of gradient calculation with backpropagation, in order to learn a good set of weights across the whole network, to achieve a purpose.

With increasingly powerful computation, and indeed, now, with specialized computer hardware that is customized to do these calculations efficiently, it now becomes possible to train such deep networks. However, such training is not always easy. There are some common challenges with deep neural networks. One source of challenge is the enormous number of degrees of freedom, in terms of the number of weights or model parameters, that exist in a DNN model. There may be millions or even billions of weights to be learned.

Thus, overfitting can be a major concern, especially if one does not have enough different and representative training samples. While there are a variety of approaches to control against over fitting, like drop out and regularization, it remains the case, that deep learning often requires enormous numbers of training samples. In addition, large models can be quite expensive to train with many epochs or cycles of training required. In addition to the large number of model weights, there can also be a large number of hyperparameters that might need to be tried, in order to get a good or optimal result. Some of these hyperparameters relate to the architecture itself.

How many filters do you use? The size of the filters, the number of layers, etc. But some of the hyperparameters might also relate to how the model itself is trained. For example, what learning rate or stopping conditions might be best. Thus, at present, it remains an art and a skill to design a good, deep, neural network for any given task. Though even there, automated neural network architecture design approaches are fast evolving. Because of the cost, both in training time and the need for very large training datasets, there's a great deal of interest in transfer learning with these systems.

In particular, there are cases where it may be possible to use a pre-trained neural network from a different or more general task, and then seek to fine-tune that model for your own particular purpose. This makes a lot of sense if we again think of early layers in a DNN as doing useful feature extraction. The hope is that those same features might be generally applicable. Then your particular application might just need to learn how to combine those features, so as to answer questions for your particular application.

Here's one realistic example. In this problem, images of auto tires during tire manufacturing are being used to try to detect and decide if there are any defects in the tire. Here, a master's student in the MIT Leaders for Global Operations Program, used the existing inception V3 classifier, as pre-trained on the large Imagenet database of images. The student kept just the first eight to 10 layers of that DNN, and discarded the later layers, replacing them with his own fully connected and classifier layers. A very good defect detector and classifier was obtained, with only about a 1,000 existing tire defect images.

Video 8: Recurrent Neural Networks (RNNs) (04:39)

Just as CNNs are an architecture that is largely motivated by image data, there are also customized neural network architectures that have been developed to deal with sequence data. Recurrent neural network, or RNN structures, enable learning of sequence-to-sequence mappings.

For example, in speech recognition, one might have an input sequence of sounds or phonemes as input, and the AI model learns to output a sequence of words or sentences. Similarly, language translation, might take a sequence of words in one language, perhaps, French. And produce a corresponding English sentence, where naturally the order, or the number of words in each sequence may be very different. The simplest conceptual RNN structure is pictured here.

The core idea is that inside the model some representation of the current state is modeled. Call that h , at each time step or sequence step, an input x_t is received. That might be a vector with multiple features. Based on the previous state, and the new input, a new state, h_t , is calculated. For example, that might be a single-layer neural network with which U , multiplying the input x_t and weights V , multiplying the previous state, summing those together, and followed by some non-linear activation function, f . In addition, another neural network component is learned to model the output vector, o , for that time step, o_t , as a function of the current state, with its own weights, W , and activation function, g .

What makes this structure interesting, and different from a simple feed-forward neural network, is that the output from parts of the network are fed back into part of the network. This feedback, or looped structure, essentially means that the network conceptually looks like a very long or even infinitely long, feed-forward multi-layer network, but where the weights are shared, or the same, for each layer.

With that structure or constraint, again, training with supervised learning and gradient descent, we can learn those weights. This feedback structure, where the current state depends on the history or sequence of previous inputs, is what enables the modeling of sequence-to-sequence data. Various RNN structures have been developed that can do a very good job at modeling of sequential data. There can be challenges in training, however.

One issue is known as vanishing or exploding gradients. Because the effective length of the sequence can be very long, gradients accumulate across hundreds of layers. That is to say, hundreds of points of time in the past. And those can become extremely small or extremely large. Approaches both in training, and in the RNN architecture itself have developed to combat this. The so-called LSTM, or long short-term memory is an RNN with a particular internal state in neural network structure that adds learned gating functions to tell the network what portions of the past state, or inputs or outputs, are most relevant to achieve the learning goal.

This enables the LSTM to learn dynamics at multiple time scales. For example, perhaps there are monthly periodic effects, in addition to more recent daily effects, that are important in predicting an output. Other RNN structures have been developed that are sometimes more complicated or sometimes simpler. For example, the gated recurrent unit, or GRU, is an RNN that is like an LSTM,

but does not have internal output gates. This simpler structure sometimes does quite well and can be simpler or faster to train.

Video 9: Machine Learning and Deep Learning Summary (01:52)

Having just covered RNNs, we have come to the end of our whirlwind tour of machine learning methods. We started our introduction to machine learning with discussion of a small taxonomy or categories of machine learning methods.

And to convey just how various, and how many different methods we've actually touched on, I have highlighted the specific methods that we've talked about. The key message to emphasize again is that there are powerful neural network and deep learning methods, but also a wide range of other machine learning methods available that can be key components of AI products.

We focused on deep learning with neural networks, including MLPs, CNNs, and RNNs. And previously, we focused on a wide range of ML methods that are not neural networks including linear classifiers, decision trees, support vector machines for classification problems, and a variety of structures for regression problems.

We also covered basic approaches, like clustering, to get a sense of structure in a dataset. In exposing you to these methods, we hope you have some basic intuition now about what these AI methods do and how they work. And finally, we introduced you to some key issues and techniques at work in machine learning, including gradient descent and the difference between training, validating, and testing, in order to have models that will perform well when deployed in AI products that you might develop.